

Vision-Based Gripper Aperture Estimation

Sidharth Padmanabhan

Robotics Project

Università della Svizzera italiana (USI)

Lugano, Switzerland

Abstract—Estimating a robotic gripper’s opening distance from vision alone eliminates the need for joint encoders or dedicated proximity sensors, which may be unavailable or noisy on certain platforms. This project presents a lightweight convolutional neural network (CNN) regressor trained entirely on images captured in the CoppeliaSim physics simulator, requiring no real-robot data. The model takes a single RGB image as input and outputs a normalised aperture value in $[0, 1]$, corresponding to a physical fingertip gap of 0–87 mm on the DJI RoboMaster EP platform. A complete data-collection pipeline combines a gripper-sweep ROS 2 node with an autonomous wall-avoidance navigation controller, enabling diverse background variety without manual robot operation. Training-time augmentation (brightness/contrast jitter, Gaussian noise, bottom-half cropping) bridges the simulation-to-reality gap. Live inference runs on the robot’s onboard camera feed with simultaneous ground-truth overlay for real-time evaluation.

Index Terms—gripper aperture estimation, regression CNN, sim-to-real, ROS 2, CoppeliaSim, RoboMaster EP, autonomous navigation

I. INTRODUCTION

Robotic manipulation demands precise knowledge of the end-effector state. Vision-based estimation is attractive because it is encoder-independent, requires no wiring changes, and naturally generalises to scenarios where occlusion or mechanical wear degrades encoder reliability.

The DJI RoboMaster EP robot [1] includes a parallel-jaw gripper with a maximum fingertip separation of 87 mm, a front-facing RGB camera, and four time-of-flight (TOF) distance sensors. Our goal is to estimate the gripper aperture from a single camera frame without reading any encoder data.

The main contributions of this work are:

- 1) A sim-to-real data collection pipeline using CoppeliaSim, ROS 2, and an autonomous wall-avoidance navigator for background diversity (Section III).
- 2) A lightweight regression CNN trained exclusively on simulation data with augmentation strategies targeting lighting variation (Section V).
- 3) A live inference system that streams predictions in real time alongside a ground-truth overlay for direct evaluation (Section VI).

The complete source code for the data-collection pipeline, model training, and live inference node is publicly available at https://github.com/Sid-WC121/gripper_aperture_estimator.

II. SYSTEM OVERVIEW

A. Hardware Platform

The DJI RoboMaster EP [1] is a wheeled educational robot equipped with:

- a front-facing 1080p RGB camera (used for both data collection and live inference);
- a parallel-jaw gripper with a maximum fingertip gap of $d_{\max} = 87$ mm, driven by a prismatic joint with range $[-0.0229, +0.00098]$ m;
- four TOF distance sensors (front-left, front-right, rear-left, rear-right) with $\approx 20^\circ$ half-angle coverage.

B. Software Stack

All software runs under **ROS 2 Humble** on macOS with **CoppeliaSim 4.10.0** as the physics simulator. The `robomaster_ros` driver bridges the DJI SDK to standard ROS 2 topics. Python packages are managed by **pixi** (conda + pip), which installs PyTorch, OpenCV, and `cv_bridge` into a single reproducible environment.

C. Aperture Convention

The gripper state is described by three equivalent quantities (Table I).

TABLE I: Aperture quantity conventions.

Quantity	Range	Definition
<code>aperture_m</code>	$[-0.023, +0.001]$	Raw prismatic joint position
<code>aperture_norm</code>	$[0, 1]$	$(q - q_{\min}) / (q_{\max} - q_{\min})$
Aperture (mm)	$[0, 87]$ mm	$\text{norm} \times 87$

The CNN predicts `aperture_norm`. Physical millimetres are recovered as $d = \hat{y} \times 87$ mm. Note that `aperture_m` $\times 1000$ does *not* yield the physical gap; it gives the raw joint position in mm (≈ 24 mm range), not the fingertip separation.

III. DATA COLLECTION PIPELINE

A. Ground-Truth Labelling

Ground-truth aperture is obtained via the **CoppeliaSim ZMQ Remote API**, which queries the prismatic joint position directly from the simulation engine at 20 Hz. This yields smooth, continuous values throughout the gripper motion, unlike `/rm0/joint_states` which only updates at action boundaries (fully open or fully closed).

The `data_collector` ROS2 node captures camera frames at 10 Hz, labels each with the ZMQ joint position, and writes JPEG images plus a CSV to disk (Algorithm 1).

Algorithm 1 Data Collector (10 Hz timer callback)

- 1: Connect to CoppeliaSim ZMQ API (on first call)
 - 2: $\text{img} \leftarrow \text{subscribe}(/rm0/camera/image_color)$
 - 3: $q \leftarrow \text{sim.getJointPosition}(\text{prismatic_joint})$
 - 4: $\hat{a} \leftarrow (q - q_{\min}) / (q_{\max} - q_{\min})$
 - 5: $\hat{a} \leftarrow \min(\max(\hat{a}, 0), 1)$
 - 6: Write img to JPEG; append $(\text{filename}, q, \hat{a})$ to CSV
-

B. Static Data Collection

The `sweep_gripper` node moves the arm to a fixed position ($x=0.15\text{ m}$, $z=0.05\text{ m}$) and repeatedly opens and closes the gripper while the robot remains stationary. This produces uniform foreground images with a static background.

C. Roaming Data Collection

To increase background diversity, the `sweep_and_roam` node combines gripper sweeping with autonomous wall-avoidance navigation, allowing the robot to drive around the environment while the gripper continuously cycles.

Navigation operates as a 20 Hz timer-driven state machine:

- **FORWARD:** drive at $v=0.2\text{ m/s}$ until a front TOF sensor reads $< 0.3\text{ m}$.
- **BACKUP:** reverse at $v=-0.2\text{ m/s}$ until front clearance $> 0.2\text{ m}$.
- **ROTATING:** spin at $\omega=0.8\text{ rad/s}$ in a random direction until front clearance $> 0.5\text{ m}$, then return to FORWARD.

The two nodes share one process but use separate executors: `ExploreNode` runs on the main `MultiThreadedExecutor`; `GripperSweepNode` runs on a private `SingleThreadedExecutor` in a daemon thread, avoiding the `ValueError: generator already executing conflict` that arises when a blocking `spin_until_future_complete` is called inside a callback serviced by a shared executor.

IV. DATA AUGMENTATION

A. Bottom-Half Cropping

The RoboMaster camera mounts above the gripper, so the gripper consistently occupies the lower half of the frame. The upper half (ceiling, walls, robot body) carries no discriminative information for aperture estimation and introduces avoidable background variance. Every image is therefore cropped to its bottom half before resizing to 128×128 :

$$\text{img} \leftarrow \text{img}[\frac{H}{2} :, :] \quad \text{then resize to } 128 \times 128.$$

This crop is applied identically during training, validation, and live inference.

B. Photometric Augmentation (Train Only)

To simulate varying lighting conditions without re-collecting data, stochastic transforms are applied to training images only (Table II).

TABLE II: Training augmentation parameters.

Transform	Distribution	Effect
Brightness	$\mathcal{U}(0.6, 1.4)$	Dim and overexposed scenes
Contrast	$\mathcal{U}(0.7, 1.3)$	Flat and high-contrast scenes
Gaussian noise	$\mathcal{N}(0, 0.02^2)$	Simulated sensor noise

Validation and test sets receive no augmentation to ensure unbiased evaluation.

V. CNN ARCHITECTURE AND TRAINING

A. Architecture

We use **ImprovedRegressionCNN**, a compact encoder-regressor (Table III). The encoder comprises four convolutional blocks, each with batch normalisation for stable training. The regressor head contains two hidden layers with ReLU activations and a dropout of 0.3 to regularise against overfitting on the limited sim dataset.

TABLE III: ImprovedRegressionCNN architecture. Input: $3 \times 128 \times 128$.

Layer	Operation	Ch./Units	Output
Conv-1	Conv 3×3 , BN, ReLU, MaxPool 2	16	$16 \times 64 \times 64$
Conv-2	Conv 3×3 , BN, ReLU, MaxPool 2	32	$32 \times 32 \times 32$
Conv-3	Conv 3×3 , BN, ReLU, MaxPool 2	64	$64 \times 16 \times 16$
Conv-4	Conv 3×3 , BN, ReLU, MaxPool 2	128	$128 \times 8 \times 8$
Flatten	—	—	8,192
FC-1	Linear, ReLU, Dropout 0.3	128	128
FC-2	Linear, ReLU, Dropout 0.3	32	32
FC-3	Linear	1	1

B. Training Protocol

- **Loss:** Smooth L1 Loss (Huber loss) on `aperture_norm`.
- **Optimiser:** Adam, learning rate 10^{-3} .
- **Epochs:** 15.
- **Batch size:** 32.
- **Dropout:** 0.3 (applied after FC-1 and FC-2).
- **Split:** 70 % train / 15 % validation / 15 % test (stratified by aperture quartile).

VI. LIVE INFERENCE SYSTEM

The `live_predictor` ROS2 node streams predictions at camera frame rate (Fig. 1). It subscribes to `/rm0/camera/image_color`, applies the same bottom-half crop and normalisation as during training, runs the CNN forward pass, and overlays the result on the video feed. Ground truth is read via the ZMQ API at 20 Hz (same path as the data collector), giving smooth continuous values during gripper motion. If ZMQ is unavailable, the node falls back to `/rm0/joint_states`.

The display overlays the CNN prediction (green bar) and ground truth (blue bar) on the camera feed, alongside numeric readouts in normalised and physical units, a rolling 150-frame time-series chart, and colour-coded error status (**GOOD** $< 5\%$, $< 10\%$, **HIGH ERROR** $\geq 10\%$).

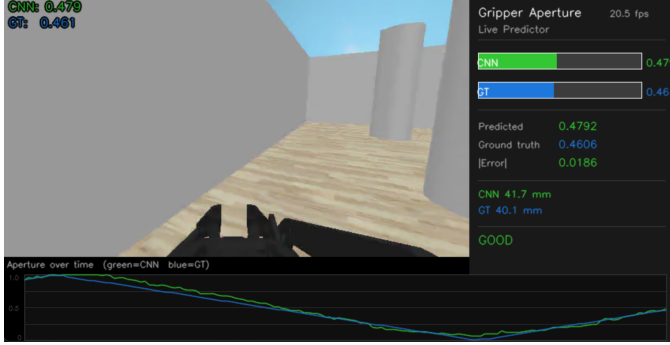


Fig. 1: Live inference on the robot's camera feed. The overlay shows the CNN prediction (green) and ground truth (blue) bars, numeric aperture values, and a rolling time-series comparison.

VII. RESULTS

A. Training and Validation Loss

Fig. 2 shows the training and validation loss over training epochs. Validation loss tracks training loss closely, indicating no significant overfitting despite the limited dataset size.

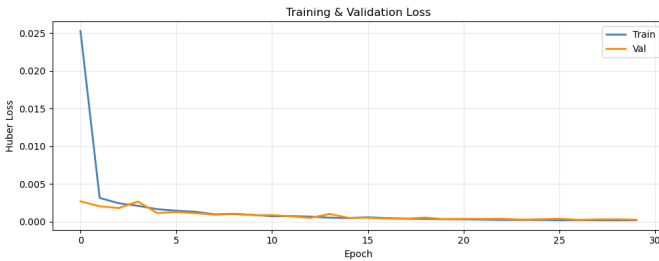


Fig. 2: Training and validation loss curves.

B. Regression Analysis

Fig. 3 shows scatter plots of ground-truth vs. predicted aperture_norm on the test set. Points close to the diagonal indicate accurate predictions. Minor deviation at extreme values (near 0 or 1) is expected due to the saturation of the gripper mechanism at its limits.

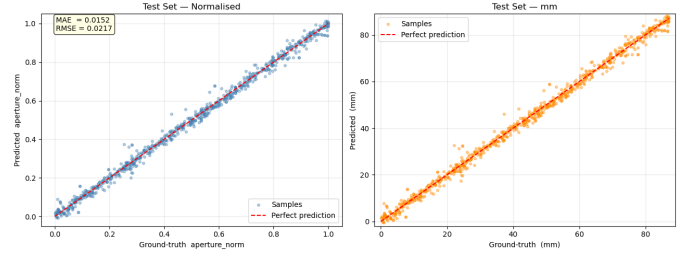


Fig. 3: Regression scatter plots on the test set. Left: normalised units. Right: physical units (mm). The red dashed line represents perfect prediction. MAE and RMSE are annotated in each subplot.

C. Test Set Performance

TABLE IV: Test-set evaluation metrics (MAE and RMSE measured on normalised aperture predictions; Huber loss used as training criterion).

Metric	Value
MAE	0.0152
RMSE	0.0217

D. Sample Predictions

Fig. 4 shows randomly selected validation images with their ground-truth and predicted aperture values in mm, using the bottom-half-cropped inputs as presented to the CNN.

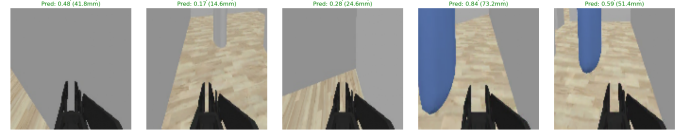


Fig. 4: Sample validation images with ground-truth and predicted aperture values (mm).

VIII. DISCUSSION

a) *Sim-to-real gap.*: The model is trained entirely on CoppeliaSim images. Real-robot deployment may suffer from texture and lighting differences. The photometric augmentation (Table II) mitigates this, but fine-tuning on a small set of real images with caliper ground truth is the recommended next step.

b) *Bottom-half crop.*: Restricting the CNN input to the gripper region substantially reduces the influence of background variation introduced during roaming data collection. However, it assumes the gripper remains in the lower half of the frame at all times, which may fail if the arm is repositioned significantly.

c) *Sensor blind spots.*: The wall-avoidance navigation uses only the two front TOF sensors. Narrow cylindrical obstacles whose diameter falls within the $\approx 20^\circ$ inter-sensor gap may not be detected until contact. A denser sensor array or occupancy-grid fusion would address this.

d) Dataset diversity.: Roaming data collection significantly improves background variety compared to static collection. Future work could add random object placement in the simulation scene for further diversity.

IX. CONCLUSION

This project presented a vision-based pipeline for estimating the aperture of a parallel-jaw gripper on the DJI RoboMaster EP using a regression CNN trained exclusively on simulation data. An autonomous roaming data-collection node provides background-diverse training images. Photometric augmentation and bottom-half cropping reduce the simulation-to-real gap without additional data collection. The live inference node enables direct, real-time comparison of CNN predictions against ground-truth values during operation.

Future work includes fine-tuning on real-robot images to close the remaining simulation-to-reality gap, integrating additional sensors for improved wall-avoidance coverage, and extending the approach to continuous estimation of the full arm configuration.

REFERENCES

- [1] DJI, “RoboMaster EP Core,” <https://www.dji.com/robomaster-ep-core>, 2021.